

MODUL 18 - JOBSHEET (Kegiatan Belajar 18)**Pemanggilan Halaman dengan PHP****PETUNJUK PENGGUNAAN MODUL**

1. Sebelum mengerjakan Modul ini, siswa harus sudah memahami tentang Dasar-dasar PHP dan perintah MySQL.
2. Baca dan pahami **Dasar Teori** yang telah diberikan oleh guru pengajar di modul ini.
3. Kerjakan / cobalah **contoh-contoh program** yang ada di **Dasar Teori** agar lebih paham.
4. Kerjakan **Tagihan Belajar** yang terdiri dari **Latihan-latihan** dan **Tugas Mandiri** yang ada di modul ini untuk **dikumpulkan**.

TUJUAN

- ✓ Memahami cara pemanggilan halaman menggunakan PHP
- ✓ Menyiapkan kerangka halaman yang akan dibuat (halaman admin)
- ✓ Menampilkan halaman yang akan dipanggil

DASAR TEORI

Pemanggilan halaman dalam PHP merupakan teknik untuk menampilkan konten yang berbeda dalam satu file utama dengan memanfaatkan parameter pada URL, biasanya menggunakan metode `GET`. Teknik ini sering digunakan dalam pembuatan sistem berbasis web seperti portal berita atau admin panel untuk mempermudah navigasi tanpa harus membuat banyak file terpisah sebagai halaman utama.

Dalam PHP, metode `GET` digunakan untuk mengambil nilai parameter yang dikirim melalui URL. Parameter tersebut kemudian digunakan untuk menentukan halaman mana yang akan ditampilkan. Contohnya adalah penggunaan `$_GET['page']` untuk mengambil nilai dari URL seperti:

	<code>index.php?page=dashboard</code>
--	---------------------------------------

Nilai `dashboard` kemudian digunakan untuk memanggil file tertentu menggunakan fungsi `include` atau `require`.

Fungsi `include` dalam PHP digunakan untuk menyisipkan file lain ke dalam file utama saat program dijalankan. Dengan menggunakan teknik ini, struktur aplikasi menjadi lebih modular karena setiap halaman dapat dipisahkan ke dalam file yang berbeda, seperti halaman dashboard, user, atau laporan.

Contoh implementasi sederhana:

```
$page = isset($_GET['page']) ? $_GET['page'] : 'dashboard';  
include "pages/$page.php";
```

Namun, penggunaan pemanggilan halaman secara langsung dari parameter `GET` memiliki risiko keamanan, seperti **Local File Inclusion (LFI)**, yaitu celah yang memungkinkan pengguna mengakses file yang tidak seharusnya ditampilkan. Oleh karena itu, diperlukan pembatasan akses halaman menggunakan teknik **whitelist**, yaitu hanya mengizinkan halaman tertentu yang boleh dipanggil.

Contoh pengamanan:

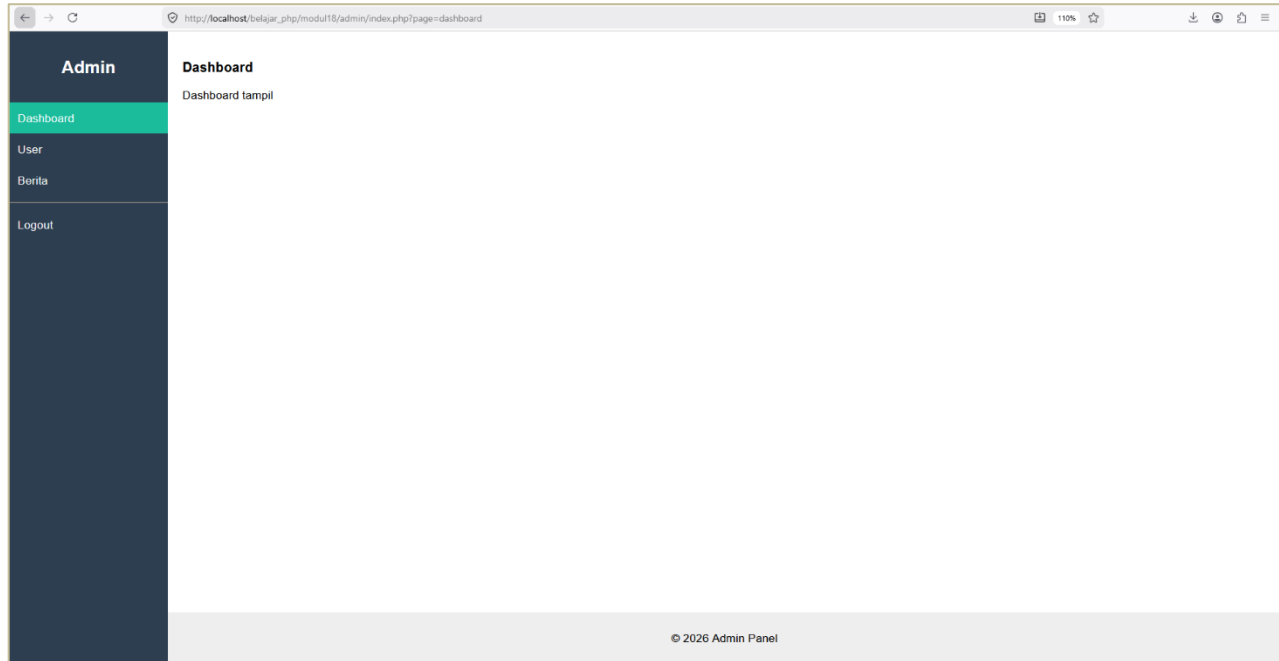
```
if (!in_array($page, $allowed)) {  
    $page = 'dashboard';  
}  
  
include "pages/$page.php";
```

Dengan adanya pembatasan tersebut, sistem menjadi lebih aman dan terhindar dari akses file yang tidak diinginkan.

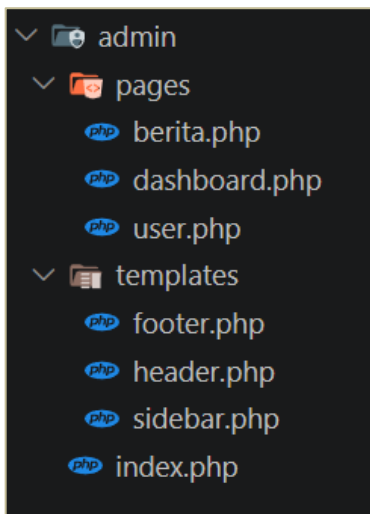
Selain itu, teknik ini juga mendukung konsep **modular programming**, di mana setiap bagian sistem dipisahkan berdasarkan fungsi tertentu sehingga memudahkan dalam pengembangan, pemeliharaan, dan pengelolaan kode program.

LATIHAN :

Dalam latihan kali ini, kita akan membuat sebuah simulasi pemanggilan halaman pada halaman admin dengan tampilan seperti ini :



Buat struktur folder di htdocs :



- 1. Buat Database dan Tabel** (bisa menggunakan database sebelumnya (tb_user dan tb_berita))

2. File Utama (index.php)

```
1  <?php
2  // menentukan halaman default
3  $page = isset($_GET['page']) ? $_GET['page'] : 'dashboard';
4
5  // whitelist halaman (biar aman)
6  $allowed = ['dashboard', 'user', 'berita'];
7
8  // cek apakah halaman diizinkan
9  if (!in_array($page, $allowed)) {
10 |     $page = 'dashboard';
11 | }
12 ?>
13
14 <?php include 'templates/header.php'; ?>
15 <?php include 'templates/sidebar.php'; ?>
16
17 <div class="main">
18 |     <div class="content">
19 |         <?php include "pages/$page.php"; ?>
20 |     </div>
21
22 |     <?php include 'templates/footer.php'; ?>
23 </div>
```

url yang dihasilkan :

Dashboard : http://localhost/belajar_php/modul18/admin/index.php?page=dashboard

Halaman User : http://localhost/belajar_php/modul18/admin/index.php?page=user

Halaman Berita : http://localhost/belajar_php/modul18/admin/index.php?page=berita

3. header.php (CSS layout sidebar)

```
<!DOCTYPE html>
<html>
<head>
  <title>Admin Panel</title>
  <style>
body {
  margin: 0;
  font-family: Arial;
  display: flex;
}

/* Sidebar */
.sidebar {
  width: 220px;
  background: #2c3e50;
  color: white;
  min-height: 100vh;
}

.sidebar h2 {
  text-align: center;
  padding: 15px 0;
}

.sidebar a {
  display: block;
  padding: 12px;
  color: white;
  text-decoration: none;
}

.sidebar a:hover {
  background: #34495e;
}

.active {
  background: #1abc9c;
}
```

Lanjutan >>>

```
/* Main layout */
.main {
  flex: 1;
  display: flex;
  flex-direction: column;
  min-height: 100vh;
}

/* Content */
.content {
  flex: 1;
  padding: 20px;
}

/* Footer */
.footer {
  background: #eee;
  padding: 10px;
  text-align: center;
}
</style>
</head>
<body>
```

4. sidebar.php (menu kiri seperti AdminLTE)

```
1 <div class="sidebar">
2   <h2>Admin</h2>
3
4   <a href="index.php?page=dashboard"
5     class="<?= ($page=='dashboard') ? 'active' : '' ?>">Dashboard</a>
6
7   <a href="index.php?page=user"
8     class="<?= ($page=='user') ? 'active' : '' ?>">User</a>
9
10  <a href="index.php?page=berita"
11    class="<?= ($page=='berita') ? 'active' : '' ?>">Berita</a>
12
13  <hr>
14
15  <a href="logout.php">Logout</a>
16 </div>
```

5. footer.php

```
1 <div class="footer">
2 |   <p>© 2026 Admin Panel</p>
3 </div>
4 </body>
5 </html>
```

6. pages/dashboard.php

```
1 <h3>Dashboard</h3>
2 <p>Dashboard tampil</p>
```

7. pages/user.php

```
1 <h3>Manajemen User</h3>
2 <p>Data user ditampilkan di sini</p>
```

8. pages/berita.php

```
1 <h3>Berita</h3>
2 <p>Data berita ditampilkan di sini</p>
```

Hasilnya

- ✓ Sidebar di kiri (fix seperti AdminLTE)
- ✓ Konten di kanan
- ✓ Menu aktif otomatis highlight
- ✓ Routing tetap pakai ?page=
- ✓ Footer selalu muncul di bawah
- ✓ Kalau konten sedikit → footer tetap di bawah layar
- ✓ Kalau konten panjang → footer ikut turun

TUGAS MANDIRI :

- ✓ Lanjutkan Latihan dengan menambahkan proses **Insert, Detail, Edit** dan **Delete** untuk menu User dan berita (yang sudah dikerjakan pada modul sebelumnya)
- ✓ Gunakan konsep session untuk autentikasi user

Halaman Login User

Level admin : masuk ke halaman/dashboard admin dan dapat melakukan/mengakses halaman insert, update, delete, dan Detail data

Level user : masuk ke halaman/dashboard user dan hanya dapat mengakse halaman Detail/melihat saja.

Logout session destroy dan kembali ke halaman login lagi

Catatan :

Gunakan Layout css yang menarik dan bagus (boleh menggunakan bootstrap)

Jika belum faham tentang penggunaan session untuk autentifikasi user dan penggunaan bootstrap, carilah literatur/tutorial di internet.